```
import javax.validation.constraints.Positive;

public record Book (

  @NotBlank(message = "The book ISBN must be defined.")
  @Pattern(                                                    ❶
    regexp = "^([0-9]{10}|[0-9]{13})$",
    message = "The ISBN format must be valid."
  )
  String isbn,

  @NotBlank(                                                   ❷
    message = "The book title must be defined."
  )
  String title,

  @NotBlank(message = "The book author must be defined.")
  String author,

  @NotNull(message = "The book price must be defined.")
  @Positive(                                                   ❸
    message = "The book price must be greater than zero."
  )
  Double price
){}
```

❶ The annotated element must match the specified regular expression (standard ISBN format).

❷ The annotated element must not be null and must contain at least one non-whitespace character.

❸ The annotated element must not be null and must be greater than zero.

NOTE Books are uniquely identified by their ISBN (International Standard Book Number). ISBNs used to be composed of 10 digits, but they now consist of 13. For simplicity, we'll limit ourselves to checking for their length and whether all the elements are digits by using a regular expression.

The annotations from the Java Bean Validation API define the constraints, but they are not enforced yet. We can instruct Spring to validate the `Book` object in the `BookController` class by using the `@Valid` annotation whenever a `@RequestBody` is specified as a method argument. In this way, whenever we create or update a book, Spring will run the validation and throw an error if any constraint is violated. We can update the `post()` and `put()` methods in the `BookController` class as follows.

Listing 3.13 Validating books passed in a request body